Institut Supérieur des Études Technologiques de Mahdia

Département : Technologies de l'Informatique

Développement des Systèmes Informatiques — DSI3.2

Rapport de Projet de Fin d'Études

Koda — Robot compagnon intelligent et émotionnel connecté, piloté par une application cross-platform

Système Embarqué · Intelligence Artificielle · Application Mobile Cross-Plateforme

Réalisé par :

Oussema KARIA

Salahedin JENDOUBI

Encadré par :

M.Wahid BANOUR — (Encadrant)

M.Hamza SASSI (Encadrant Entreprise)

Organisme d'accueil : —

Période : Février 2026 – Juin 2026

Abstract

This report presents KODA, an intelligent embedded life-assistant system designed to provide personalized support and companionship to users through a virtual assistant with its own unique nature and personality. KODA is capable of responding intelligently to any person, reasoning like a human, and recognizing individuals through its security and configuration modes.

The system is built around a multi-component architecture: a Raspberry Pi embedded system serving as the robot's physical core; an n8n workflow engine forming the brain of KODA, responsible for preparing and reasoning through responses; a Python service layer acting as a universal distributor, communicating with n8n, the database, the mobile application, and the Raspberry Pi while exposing all ecosystem endpoints; a cross-platform mobile application for configuration and life-monitoring; and a persistent database serving as KODA's memory. Developed following an agile methodology, the project ensures seamless integration between embedded hardware and scalable software components.

Keywords: Embedded System, Virtual Assistant, Raspberry Pi, n8n, Python, Cross-Platform, Artificial Intelligence, Life Assistant, KODA.

Résumé

Ce rapport présente KODA, un système embarqué intelligent conçu pour être un assistant de vie et un ami virtuel doté de sa propre personnalité et nature. KODA est capable de répondre de manière réfléchie à n'importe quelle personne, de raisonner comme un être humain et de reconnaître les individus grâce à ses modes de sécurité et de configuration.

L'architecture du système repose sur plusieurs composants complémentaires : un système embarqué Raspberry Pi constituant le cœur physique du robot ; un moteur de workflows n8n formant le cerveau de KODA, responsable de la préparation et de la réflexion des réponses ; une couche de services Python agissant comme distributeur universel, assurant la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi tout en exposant les endpoints de tout l'écosystème ; une application mobile cross-plateforme pour la configuration et le suivi de vie du robot ; et une base de données persistante servant de mémoire à KODA.

Mots-clés : Système Embarqué, Assistant Virtuel, Raspberry Pi, n8n, Python, Cross-Plateforme, Intelligence Artificielle, Assistant de Vie, KODA.

DÉDICACE

Je dédie ce travail à ma famille, pilier silencieux de ma force et source inépuisable de soutien. Merci pour vos sacrifices immenses et pour avoir cru en moi tout au long de mon parcours académique.

À mes parents, pour leur amour inconditionnel, leurs encouragements et leurs prières qui ont été la lumière guidant chacun de mes pas. Ce succès est le vôtre autant que le mien.

À mes amis et collègues, pour leur soutien moral et pour avoir partagé avec moi cette aventure inoubliable.

À mon binôme Salahedin, pour sa persévérance, sa créativité et sa complicité tout au long de ce projet.

À tous ceux qui ont contribué, même par un simple mot d'encouragement, à l'accomplissement de ce travail.

Oussema KARIA

DÉDICACE

À ma famille,
pour son soutien inconditionnel et ses sacrifices immenses
tout au long de mon parcours académique.

À mon binôme Oussema, pour son enthousiasme, sa rigueur
et sa précieuse collaboration dans la réalisation de ce projet.

À mes encadrants,
pour leur guidance, leur patience et leur expertise technique.

Salahedin JENDOUBI

Acknowledgements

Nous tenons à exprimer notre profonde et sincère gratitude à toutes les personnes qui ont contribué à la réalisation et au succès de ce projet.

Nous adressons nos remerciements les plus chaleureux à notre encadrant académique pour ses précieux conseils, sa disponibilité constante et son suivi rigoureux, garant de l'intégrité académique de notre travail.

Nous remercions également notre encadrant entreprise ainsi que toute l'équipe de l'organisme d'accueil pour leur accueil chaleureux, la mise à disposition des ressources nécessaires et les éclairages techniques partagés tout au long de notre stage.

Nous remercions par ailleurs la direction et le corps enseignant de l'ISET MAHDIA, département DSI, pour la formation de qualité qu'ils nous ont dispensée.

Enfin, nous remercions sincèrement nos amis et collègues pour leur soutien moral et pour avoir partagé avec nous cette aventure inoubliable.

CONTENTS

Introduction Générale **1**

I Cadre Général du Projet et Présentation de l'Organisme **2**

I.1 Introduction du chapitre 3

Introduction du chapitre 3

I.2 Contexte Général du Projet 3

I.3 Host Organization Overview (MicroEdition) 3

I.3.1 Company Profile 3

I.3.2 Services 4

I.3.3 Research & Development (R&D) 4

I.3.4 Organizational Structure 4

I.4 Problématique 5

I.5 Analyse des Solutions Existantes 5

I.5.1 Étude de l'existant 6

I.5.2 Synthèse et Critique : Le "Manque d'Humanité" 6

I.5.3 La Différenciation de Koda 7

I.6 Solution Proposée 7

I.7 Méthodologie du Projet 8

I.7.1 Approche Agile et Cycle en V 8

I.8 Conclusion 9

II Étude des Besoins et Modélisation UML **10**

III Cycle 1 : ANALYSEUR ET RÉFLÉCHISSEMENT **12**

III.1 Introduction 13

III.2 Définitions Fondamentales 13

III.2.1 n8n 13

III.2.2 Webhook 14

III.3 Fondements Théoriques 14

III.3.1 La Théorie du Double Traitement 14

III.4 Fonctionnalités Techniques 15
III.4.1 Premier Flux — Webhook 1 : Analyse et Réponse 16
III.4.1.1 Principe Général 16
III.4.1.2 Format de la Requête Entrante 16
III.4.1.3 Description Détaillée des Blocs de Traitement 16
III.4.1.3.1 Block 1 : Réception, Vérification de l'État et Contrôle d'Accès 16
III.4.1.3.2 Block 2 : Vérification de la Question dans l'Historique 17
III.4.1.3.3 Block 3 : Classification Sémantique de la Question . . 19
III.4.1.3.3.1 Composant Principal : AI Agent 20
III.4.1.3.3.2 Prompt Engineering — Instruction de Classification 20
III.4.1.3.3.3 Détail des Types de Questions Classifiés 22
III.4.1.3.4 Block 4 : Reconnaissance et Enregistrement de l'Utilisateur 22
III.4.1.3.5 Block 5 : Récupération Contextuelle et Génération de la Réponse 24
III.4.1.3.5.1 Branche Mémorielle (`use_memory = true`) . . 25
III.4.1.3.5.2 Branche Générale (`use_memory = false`) . . 27
III.4.1.4 Vue Complète du Block 5 — Les Deux Branches 29
III.4.1.4.1 Block 6 : Mise à Jour de Mémoire 31
III.4.1.5 Diagramme de Séquence du Webhook 1 34
III.4.2 Deuxième Flux — Webhook 2 : Filtre et Synthèse Journalière 34
III.4.2.1 Principe et Inspiration 34
III.4.2.2 Fonctionnement du Webhook 2 35
III.4.2.3 Structure des Fiches de Synthèse Générées 35
III.4.2.4 Impact sur le Système Global 36
III.4.3 Troisième Flux — Webhook 3 : Spontanéité et Interaction Proactive 37
III.4.3.1 Principe : La Spontanéité comme Dimension Humaine 37
III.4.3.2 Déclenchement et Fonctionnement 37
III.4.3.3 Génération de la Phrase d'Accroche 37
III.5 Vue Générale du Cycle 1 38
III.6 Conclusion 38

LIST OF FIGURES

I.1 Architecture Globale de la Solution Proposée 8

III.1 Architecture cognitive de KODA inspirée de la théorie du double traitement 15
III.2 Block 1 — Vérification de l'état opérationnel et contrôle d'accès de KODA 17
III.3 Block 1 — Diagramme de séquence du premier bloc 17
III.4 Block 2 — Vérification et comparaison avec les 5 derniers messages 18
III.5 Block 2 — Diagramme de séquence du deuxième bloc 18
III.6 Block 3 — Diagramme de séquence du troisième bloc 20
III.7 Block 3 — Classification sémantique de la question par agent IA 22
III.8 Block 4 — Reconnaissance et enregistrement du profil utilisateur 24
III.9 Block 4 — Diagramme de séquence du quatrième bloc 24
III.10 Block 5.1 — Branche mémorielle : récupération et génération contextuelle 26
III.11 Block 5.2 — Branche générale : contexte conversationnel et génération de réponse 29
III.12 Block 5 — Vue complète des deux branches de traitement contextuel 30
III.13 Block 5 — Diagramme de séquence du cinquième bloc 30
III.14 Block 5 — Diagramme de classes du cinquième bloc 31
III.15 Block 6 — Pipeline de mise à jour de la mémoire et persistance des données 33
III.16 Block 6 — Diagramme de classes du sixième bloc 34
III.17 Vue complète du deuxième workflow 35
III.18 Vue des outputs du 2éme workflow 36
III.19 Vue complète du troisiéme workflow 37

LIST OF TABLES

III.1 Correspondance entre mécanismes cognitifs humains et implémentations de KODA 38

INTRODUCTION GÉNÉRALE

Dans un monde en constante évolution technologique, la demande en solutions intelligentes capables d'améliorer la qualité de vie des individus n'a jamais été aussi forte. Les assistants virtuels et les systèmes embarqués intelligents représentent aujourd'hui une réponse concrète à ce besoin, en combinant les avancées de l'intelligence artificielle, des systèmes embarqués et du développement logiciel moderne.

Ce projet de fin d'études, réalisé dans le cadre de la formation en Développement des Systèmes Informatiques (DSI3.2) à l'ISET Mahdia, porte sur la conception et le développement de **KODA** : un assistant de vie et ami virtuel embarqué. KODA est un système robotique doté de sa propre personnalité et nature, capable de répondre intelligemment à toute personne, de raisonner à la manière d'un être humain, de reconnaître les individus à travers ses modes de sécurité, et d'être configuré et suivi via une application mobile cross-plateforme dédiée.

La problématique centrale de ce projet est : **Comment concevoir un système embarqué intelligent capable d'assurer un rôle d'assistant de vie personnalisé, intégrant reconnaissance des personnes, réflexion autonome et pilotage à distance via une application mobile ?**

Ce rapport est organisé comme suit :

- **Chapitre I** présente le cadre général du projet, l'organisme d'accueil, la problématique, l'analyse de l'existant et la solution proposée.
- **Chapitre II** définit les besoins fonctionnels et non fonctionnels ainsi que la modélisation UML du système.
- **Chapitre III** décrit l'architecture globale du système KODA et l'interaction entre ses différentes couches.
- **Chapitre IV** détaille le système embarqué Raspberry Pi, son rôle et son implémentation.
- **Chapitre V** présente le moteur de traitement n8n et la couche Python, cœur logique de KODA.
- **Chapitre VI** couvre l'application mobile cross-plateforme de configuration et de suivi.
- **Chapitre VII** traite du déploiement, de la base de données et des tests de validation du système complet.

CHAPTER I

CADRE GÉNÉRAL DU PROJET ET PRÉSENTATION DE L'ORGANISME

I.1 Introduction du chapitre

Ce chapitre esquisse le cadre fondamental de notre étude en définissant l'environnement contextuel et organisationnel du projet. Nous commençons par introduire l'organisme d'accueil, en mettant en évidence son expertise technologique et sa structure interne dédiée à l'innovation.

Ensuite, nous exposons la problématique centrale, justifiant le besoin critique d'un système intelligent. Une analyse rigoureuse des solutions existantes nous permet d'identifier les limites actuelles et de mieux positionner notre système proposé. Enfin, nous détaillons la méthodologie de projet adoptée, assurant une approche structurée et efficace tout au long du cycle de développement.

I.2 Contexte Général du Projet

L'évolution technologique actuelle transforme notre interaction avec l'environnement. La robotique sociale et les assistants personnels virtuels s'imposent comme des éléments clés pour améliorer la qualité de vie, offrant des solutions innovantes pour l'assistance quotidienne.

Le projet KODA s'inscrit dans cette dynamique en proposant un assistant de vie virtuel, conçu sous la forme d'un système embarqué intelligent. Contrairement aux solutions traditionnelles qui se limitent à l'exécution de commandes, KODA vise à offrir une véritable présence interactive, dotée d'une personnalité propre et d'une capacité de raisonnement.

La complexité de ce système réside dans l'intégration harmonieuse de multiples composants hétérogènes. Il nécessite la combinaison d'un environnement matériel embarqué (Raspberry Pi), d'un moteur de workflow intelligent (n8n) agissant comme cerveau, d'une couche logicielle orchestrant les communications (Python), et d'interfaces de contrôle intuitives (application mobile croisée). L'objectif est de créer un écosystème cohérent capable de reconnaître les utilisateurs, de traiter les requêtes de manière naturelle, et de fournir un accompagnement personnalisé.

I.3 Host Organization Overview (MicroEdition)

I.3.1 Company Profile

MicroEdition is a pioneering enterprise specializing in Information and Communication Technologies (ICT). Established in 2025, the company is strategically headquartered within the Mahdia Business Incubator. This environment provides a robust ecosystem for innovation, offering the necessary infrastructure to support high-potential technology startups. MicroEdition is dedicated to the architecture and deployment of intelligent, scalable, and secure digital solutions designed to bridge the gap between academic research and industrial application.

I.3.2 Services

The company's portfolio is built upon three fundamental technological pillars:

- **Software Development:** Crafting high-performance web and mobile applications tailored to specific business logic.
- **Internet of Things (IoT):** Designing end-to-end connected ecosystems for industrial and agricultural automation.
- **Cybersecurity:** Implementing rigorous security protocols to protect digital assets and ensure data integrity within private and public sectors.

I.3.3 Research & Development (R&D)

A distinguishing feature of MicroEdition is its specialized Research and Development division. This department focuses heavily on Edge AI and Embedded Intelligence. By investigating methodologies to execute complex neural networks locally on microcontrollers, the R&D team seeks to eliminate the dependency on constant cloud connectivity. This strategic focus is central to this project, as it enables the creation of autonomous, resilient systems capable of operating in remote agricultural environments where internet access may be intermittent.

I.3.4 Organizational Structure

MicroEdition operates through an Agile-driven framework that emphasizes rapid iteration and interdisciplinary collaboration. The company's internal structure is composed of specialized units, including:

- **Software Architects:** Designing scalable system backends.
- **Embedded Systems Engineers:** Bridging the gap between hardware and software.
- **AI and Data Scientists:** Developing and optimizing machine learning models for edge deployment.

This synergy allows MicroEdition to maintain a fast-paced innovation cycle, ensuring that every project—such as the Intelligent Hydroponic Fodder System—is built on a foundation of technical excellence and modern architectural standards.

I.4 Problématique

L'évolution constante de notre société moderne, marquée par un rythme de vie accéléré et une digitalisation croissante, a paradoxalement accentué certains besoins fondamentaux de l'être humain, notamment celui de l'accompagnement et de l'interaction sociale.

Le problème central auquel nous tentons de répondre s'articule autour de trois axes principaux :

A. Le besoin de présence et d'interaction naturelle

Malgré la prolifération des appareils connectés, la plupart des outils actuels (smartphones, enceintes connectées classiques) restent des outils purement fonctionnels. Ils exécutent des ordres mais n'offrent pas de réelle interaction "humaine". Il existe un manque flagrant de systèmes capables d'afficher une personnalité propre et de simuler une forme de réflexion empathique capable de briser la solitude ou d'assister l'utilisateur au quotidien.

B. La complexité de la gestion de vie autonome

Dans un environnement domestique, la gestion des tâches, de la sécurité et du suivi technique des objets connectés devient de plus en plus complexe pour l'utilisateur moyen. Il n'existe pas de solution centralisée qui combine à la fois un assistant de vie intelligent (capable de raisonner) et un système de sécurité actif (capable de reconnaître les personnes et de protéger l'habitat).

C. Les limites de l'intelligence statique

La majorité des assistants actuels dépendent de réponses préprogrammées et rigides. Le défi est de passer d'une machine qui "répond" à une machine qui "réfléchit" et "apprend". Le problème est donc de concevoir une architecture capable de :

- Identifier l'interlocuteur de manière sécurisée.
- Traiter des demandes complexes de manière fluide et humaine.
- Offrir une interface de contrôle simple (application mobile) pour superviser cette "vie artificielle".

En résumé, la question fondamentale de notre projet est la suivante : **Comment concevoir un compagnon robotique intelligent, doté d'une mémoire et d'une personnalité, capable d'offrir un accompagnement humain authentique tout en assurant une gestion technique et sécurisée de l'environnement de l'utilisateur ?**

I.5 Analyse des Solutions Existantes

I.5.1 Étude de l'existant

Dans cette section, nous examinons les assistants domestiques et les robots compagnons les plus populaires sur le marché afin d'identifier leurs forces et leurs lacunes.

- **A. Les Assistants Vocaux Statiques (Amazon Alexa, Google Home)**

  Ces dispositifs sont les plus répandus. Ils excellent dans l'exécution de tâches simples (domotique, rappels, recherches internet).

  Limites : Ils n'ont pas de présence physique mobile. Surtout, ils restent des outils purement réactifs : ils attendent un "mot déclencheur" et répondent de manière standardisée. Il n'y a aucune notion de "personnalité" évolutive ou de sentiment d'appartenance.

- **B. Les Robots Domestiques (ex: Amazon Astro, Vector/Cozmo)**

  Certains robots essaient d'occuper l'espace physique. Amazon Astro, par exemple, surveille la maison et suit l'utilisateur.

  Limites : Bien que mobiles, ces produits restent perçus comme des gadgets technologiques. Leur interaction est limitée par des scripts pré-enregistrés. Ils ne "réfléchissent" pas de manière humaine et leur capacité à reconnaître et s'adapter à l'humeur ou à l'identité spécifique de chaque membre de la famille de façon approfondie reste superficielle.

- **C. Les Pepper & Nao (Robotique de service)**

  Ce sont des robots haut de gamme capables de simuler des émotions et de reconnaître des visages.

  Limites : Ces solutions sont extrêmement coûteuses (plusieurs milliers d'euros) et sont destinées aux entreprises plutôt qu'au grand public. Leur configuration est complexe et ils ne sont pas conçus comme des "amis" personnels intégrés à un écosystème mobile fluide pour l'utilisateur lambda.

I.5.2 Synthèse et Critique : Le "Manque d'Humanité"

Le point commun de tous ces concurrents est leur nature générique. Ce sont des machines produites en série qui traitent chaque utilisateur de la même manière.

- **Absence de personnalité :** La réponse d'une Alexa est la même pour tout le monde. Elle n'a pas de "caractère" propre qui pourrait la faire passer pour un compagnon de vie.
- **Froideur algorithmique :** Les interactions sont basées sur des arbres de décision rigides. L'utilisateur sent qu'il parle à une base de données, pas à une entité qui "réfléchit".

- **Dépendance au Cloud propriétaire :** L'utilisateur a peu de contrôle sur la "mémoire" ou la logique interne de l'assistant.

I.5.3 La Différenciation de Koda

C'est ici que notre projet se distingue. Contrairement aux machines industrielles, Koda est conçu pour être :

- **Unique :** Grâce à l'intégration de n8n (le cœur de réflexion), Koda peut avoir une logique de réponse personnalisée et évolutive.
- **Multimodal :** Il combine la vision (Raspberry Pi), la réflexion (IA via n8n) et la mobilité.
- **Accessible :** Une application mobile dédiée permet de configurer sa "personnalité", rendant l'utilisateur maître de son compagnon.

I.6 Solution Proposée

Pour surmonter ces limitations, nous proposons KODA, une architecture résiliente et multi-couche conçue pour l'autonomie et l'intelligence. La solution comprend :

- **Contrôleur Embarqué (Raspberry Pi) :** Le cœur physique du système, gérant les composants matériels et assurant le fonctionnement du robot.
- **Moteur de Réflexion (n8n) :** Le cerveau de KODA, responsable de la préparation des réponses et du traitement intelligent des requêtes complexes.
- **Couche de Distribution (Python) :** Le système nerveux responsable de la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi, offrant les points d'accès (endpoints) de tout l'écosystème.
- **Expérience Utilisateur Mobile :** Une application cross-plateforme permettant à l'utilisateur de configurer, contrôler et suivre la vie de son compagnon robotique en temps réel.
- **Mémoire Persistante (Base de données) :** Un système de stockage permettant à KODA d'enregistrer des informations contextuelles et d'apprendre de ses interactions.

En intégrant ces composants, la solution proposée garantit une résilience opérationnelle, des interactions humaines authentiques et une expérience de gestion unifiée.

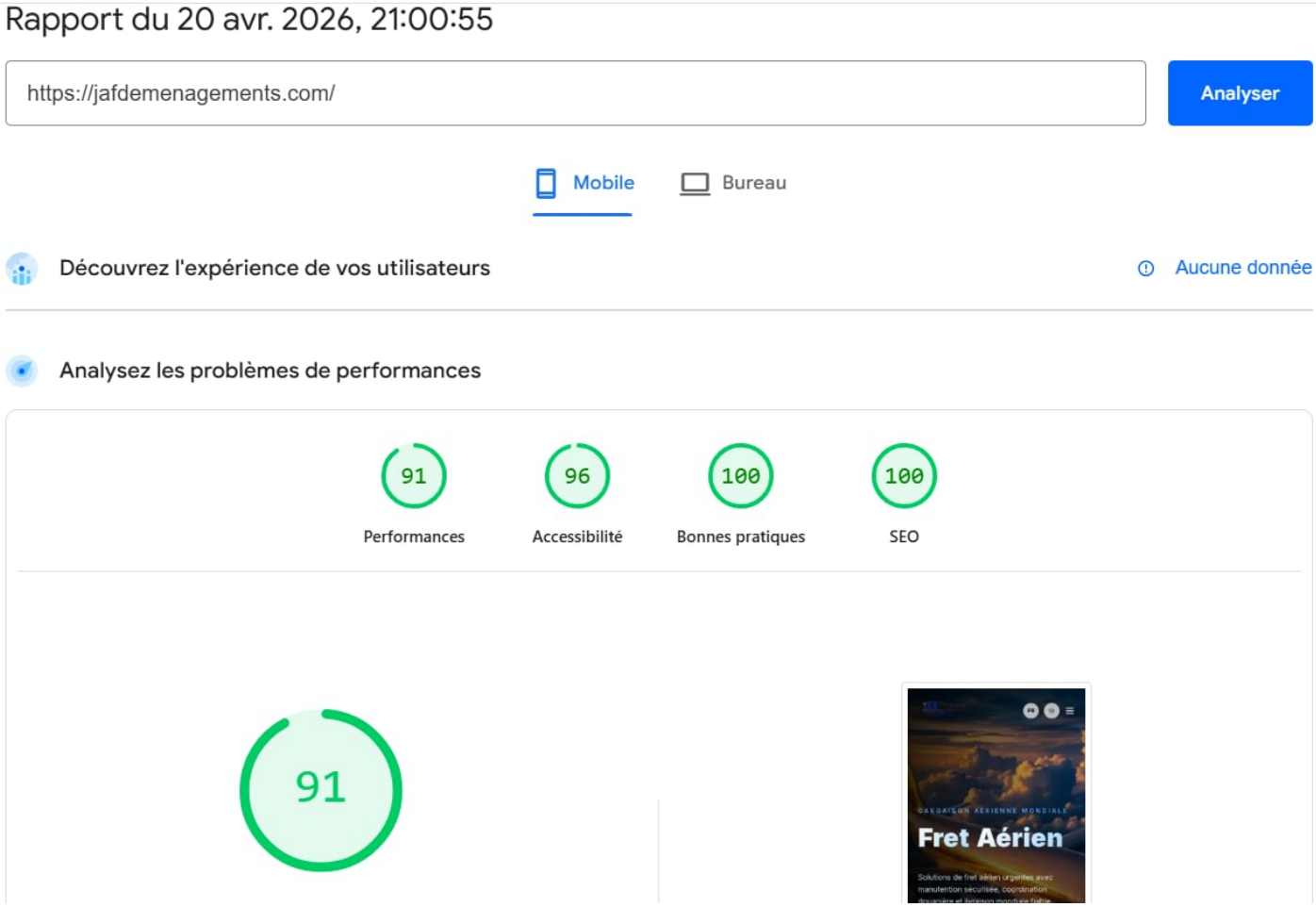

Figure I.1 – Architecture Globale de la Solution Proposée

I.7 Méthodologie du Projet

Pour garantir un haut degré de fiabilité et une intégration harmonieuse des différentes couches logicielles et matérielles, ce projet suit un cadre d'ingénierie rigoureux.

I.7.1 Approche Agile et Cycle en V

Bien que le projet intègre des concepts de prototypage rapide caractéristiques des méthodes Agiles, l'architecture globale repose sur un Cycle en V. Ce modèle représente un chemin d'exécution séquentiel où chaque phase de développement (côté gauche) est associée à une phase de test correspondante (côté droit).

Nous avons adopté cette méthodologie car elle garantit que les protocoles de test pour notre matériel embarqué et nos workflows d'intelligence artificielle soient définis avant l'assemblage physique, conduisant à un cycle d'intégration plus efficace. La figure **??** illustre les étapes fondamentales de ce cadre dans un contexte d'ingénierie général.

1) Le flux de processus : Déconstruction et Réassemblage

La méthodologie forme une forme en "V" symétrique, divisée en trois segments principaux :

- **Le segment de vérification (Étapes 1–4) :** Se concentre sur la traduction des besoins des utilisateurs en un plan architectural définitif.
- **Le sommet (Étape 5) :** L'implémentation est le tournant critique où le codage, l'assemblage matériel et la configuration des workflows ont lieu.
- **Le segment de validation (Étapes 6–8) :** Réassemble et vérifie le système à travers des tests unitaires et des tests d'intégration globaux.

2) Artefacts Clés du Projet

La traçabilité est maintenue en générant des artefacts spécifiques à chaque étape :

- **Spécifications des Besoins :** Besoins utilisateurs et techniques documentés.
- **Conception de l'Architecture Système :** Plans d'intégration matériel/logiciel.
- **Conception Détaillée des Composants :** Logique de code, endpoints API et schéma de base de données.

I.8 Conclusion

Ce chapitre a établi le contexte du projet KODA. En identifiant les lacunes critiques des assistants virtuels actuels — notamment le manque de personnalité, de réflexion autonome et d'intégration embarquée — nous avons défini notre solution innovante. De plus, en adoptant une méthodologie de développement structurée, nous avons établi une feuille de route qui garantit la fiabilité tout au long des défis techniques à venir.

Le chapitre suivant, "Étude des Besoins et Modélisation UML", initiera la première étape de ce cycle de conception en transformant les besoins de l'utilisateur en spécifications définitives et en modèles architecturaux.

CHAPTER II

ÉTUDE DES BESOINS ET MODÉLISATION UML

Ce chapitre est actuellement vide.

CHAPTER III

CYCLE 1 : ANALYSEUR ET RÉFLÉCHISSEMENT

III.1 Introduction

Ce chapitre présente le moteur central de KODA : son système d'analyse, de réflexion et de génération de réponses. L'objectif principal est de permettre au robot de répondre de manière pertinente, contextualisée et naturelle aux questions posées par les utilisateurs. KODA étant conçu comme un accompagnateur virtuel à dimension humaine, il doit comprendre non seulement le contenu des questions, mais aussi le contexte et les besoins implicites de chaque utilisateur.

Pour atteindre cet objectif, nous avons conçu une architecture logicielle organisée autour de trois flux de travail automatisés (appelés **webhooks**), chacun remplissant une fonction précise et complémentaire. Avant de détailler leur fonctionnement, nous commençons par définir les deux technologies fondamentales sur lesquelles repose l'ensemble du système : **n8n** et les **webhooks**.

III.2 Définitions Fondamentales

III.2.1 n8n

n8n (prononcé "n-eight-n", abréviation de *nodemation*) est une **plateforme d'automatisation de flux de travail open-source**, conçue pour connecter des applications, des APIs et des services entre eux sans nécessiter de développement complexe.

Techniquement, n8n fonctionne selon une architecture orientée **nœuds** (nodes) : chaque nœud représente une opération atomique — un appel à une API, une transformation de données, une condition logique, ou l'invocation d'un modèle d'intelligence artificielle. Ces nœuds sont reliés entre eux pour former des **flux de traitement** (workflows), dans lesquels les données circulent sous forme d'objets JSON structurés d'un nœud à l'autre.

Voici les caractéristiques essentielles qui ont motivé notre choix de n8n pour le projet KODA :

- **Architecture DAG (Graphe Orienté Acyclique) :** Les nœuds sont organisés en graphe sans cycle, garantissant un traitement ordonné et prévisible des données.
- **Auto-hébergement :** Contrairement à des outils comme Zapier ou Make, n8n peut être installé sur un serveur privé. Cela assure un contrôle total sur les données des utilisateurs — un critère indispensable pour un projet traitant des informations personnelles.
- **Intégration native avec les modèles IA :** n8n propose des connecteurs prêts à l'emploi pour les principaux modèles de langage (OpenAI, Anthropic Claude, Mistral), permettant d'insérer des nœuds d'intelligence artificielle directement dans le flux de traitement.
- **Interface visuelle :** Le concepteur de workflows est entièrement graphique, permettant de visualiser, tester et déboguer chaque étape du traitement en temps réel.

- **Déclencheurs multiples :** n8n supporte des dizaines de modes de déclenchement — webhooks HTTP, planification cron, événements base de données, etc.

Référence : Documentation officielle n8n — `https://docs.n8n.io`

III.2.2 Webhook

Un **webhook** est un mécanisme de communication entre deux systèmes informatiques basé sur le modèle **événement → notification → action**. Contrairement à une requête classique où c'est le client qui interroge régulièrement un serveur pour savoir si quelque chose a changé (modèle *polling*), un webhook fonctionne à l'inverse : c'est le serveur émetteur qui notifie automatiquement le serveur récepteur dès qu'un événement se produit.

Concrètement, un webhook est une **URL HTTP unique** exposée par le système récepteur (ici, n8n). Lorsqu'un événement survient dans le système émetteur (par exemple : l'utilisateur pose une question au robot), celui-ci envoie une requête HTTP POST vers cette URL, avec les données de l'événement dans le corps de la requête. Le serveur récepteur traite alors ces données et déclenche le pipeline de traitement associé.

Concept	Signification
URL du webhook	Adresse unique qui reçoit les notifications
Déclencheur	L'événement qui provoque l'envoi
Payload	Les données transmises (format JSON)
Pipeline	L'ensemble des traitements activés

Dans KODA, nous utilisons trois webhooks distincts, chacun dédié à une fonction cognitive précise du robot.

Référence : Mozilla Developer Network — `https://developer.mozilla.org/en-US/docs/Glossary/W`

III.3 Fondements Théoriques

III.3.1 La Théorie du Double Traitement

Le cadre théorique central de notre conception repose sur la **théorie du double traitement** (*Dual-Process Theory*), formalisée par le psychologue et lauréat du prix Nobel d'économie Daniel Kahneman. Cette théorie explique comment le cerveau humain traite l'information et formule ses réponses à travers deux systèmes distincts :

- **Système 1 — Rapide et Automatique :** Ce système fonctionne de manière instinctive et inconsciente. Il mobilise des associations mémorisées et des réflexes cognitifs pour répondre

quasi instantanément aux situations familières. Il gère la grande majorité de nos interactions quotidiennes sans effort apparent.

- **Système 2 — Lent et Délibéré :** Ce système entre en jeu lorsqu'une situation exige une réflexion consciente et approfondie. Il est sollicité pour les raisonnements complexes, les calculs, ou tout jugement nécessitant une analyse minutieuse. Il est plus lent, mais plus précis et rigoureux.

En pratique, face à une question, le cerveau humain commence par activer le **Système 1** : si la réponse est connue et accessible, elle est fournie immédiatement. Dans le cas contraire, le **Système 2** prend le relais pour analyser la question en profondeur et construire une réponse adaptée.

Dans KODA, nous avons transposé cette architecture cognitive : un agent de classification détermine si la question reçue relève d'un traitement rapide (accès direct à la mémoire) ou d'un traitement approfondi (analyse sémantique complète), et active le pipeline correspondant.

Référence : Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.

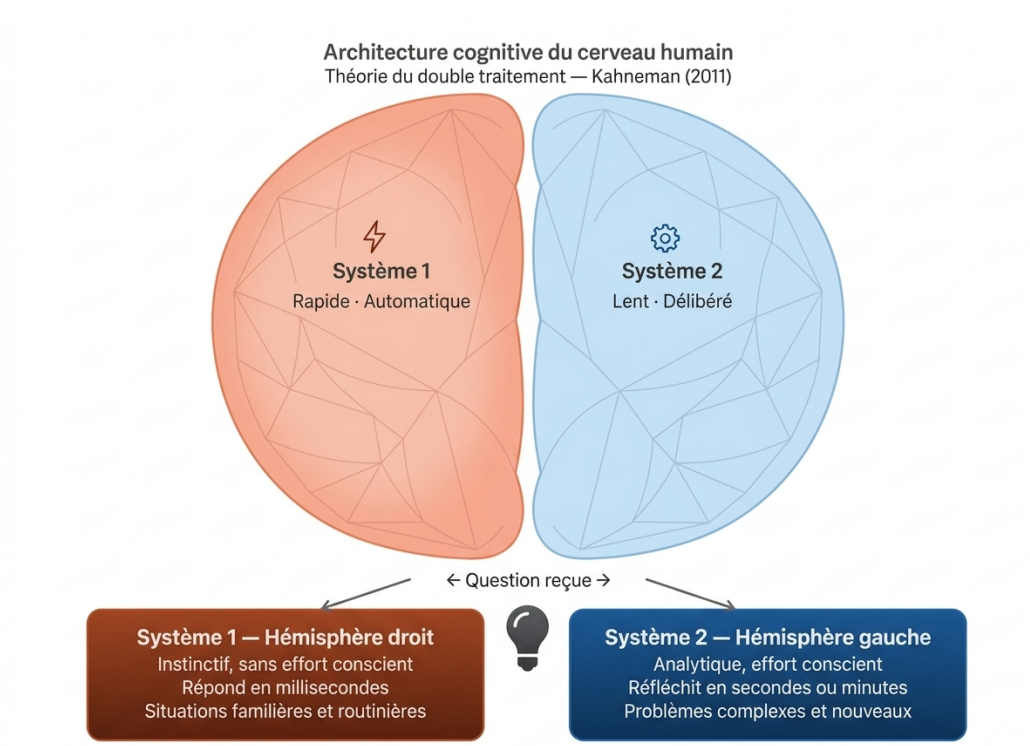

Figure III.1 – Architecture cognitive de KODA inspirée de la théorie du double traitement

III.4 Fonctionnalités Techniques

Cette section présente en détail les trois flux de travail (webhooks) qui constituent le cœur fonctionnel du système cognitif de KODA. Chaque flux remplit une fonction précise et complémentaire dans le pipeline de traitement.

III.4.1 Premier Flux — Webhook 1 : Analyse et Réponse

III.4.1.1 Principe Général

Le Webhook 1 est le pipeline principal et en temps réel de KODA. Il est déclenché à chaque fois qu'un utilisateur pose une question au robot. Son rôle est d'analyser la question, d'accéder aux informations contextuelles pertinentes, et de générer une réponse personnalisée et naturelle.

Le pipeline est composé de **6 blocs séquentiels**, chacun accomplissant une tâche précise avant de transmettre ses résultats au bloc suivant. L'ensemble du flux est orchestré par n8n.

III.4.1.2 Format de la Requête Entrante

Le webhook accepte les requêtes au format `(user_name + question)`. Le champ `user_name` est un identifiant unique généré automatiquement par le backend pour chaque utilisateur en cours d'interaction. Cet identifiant est injecté avec la question dans le flux de travail, permettant d'isoler et de gérer les données de chaque utilisateur de manière indépendante dans les environnements multi-utilisateurs.

III.4.1.3 Description Détaillée des Blocs de Traitement

III.4.1.3.1 Block 1 : Réception, Vérification de l'État et Contrôle d'Accès

À la réception de chaque requête via le **webhook**, le pipeline déclenche immédiatement la récupération de l'état actuel du robot depuis la base de données.

Cas 1 — KODA est en état ouvert (*open state*) : la requête est acceptée et transmise directement au bloc suivant pour traitement.

Cas 2 — KODA est en état fermé (*closed state*) : le système effectue un test conditionnel supplémentaire :

- Il compare le **mot de passe** fourni en entrée (*input*) avec le mot de passe attendu.
- Si la correspondance est confirmée, cela indique que l'utilisateur souhaite **ouvrir KODA** : la commande d'ouverture est alors exécutée.
- Si la correspondance échoue ou si aucune intention d'ouverture n'est détectée, le système **retourne un message indiquant que KODA est actuellement fermé** et aucune action n'est entreprise.

Ce mécanisme de garde (*gating*) combiné au contrôle d'accès par mot de passe garantit que KODA ne peut être activé que par une requête autorisée, assurant ainsi la stabilité et la sécurité du système.

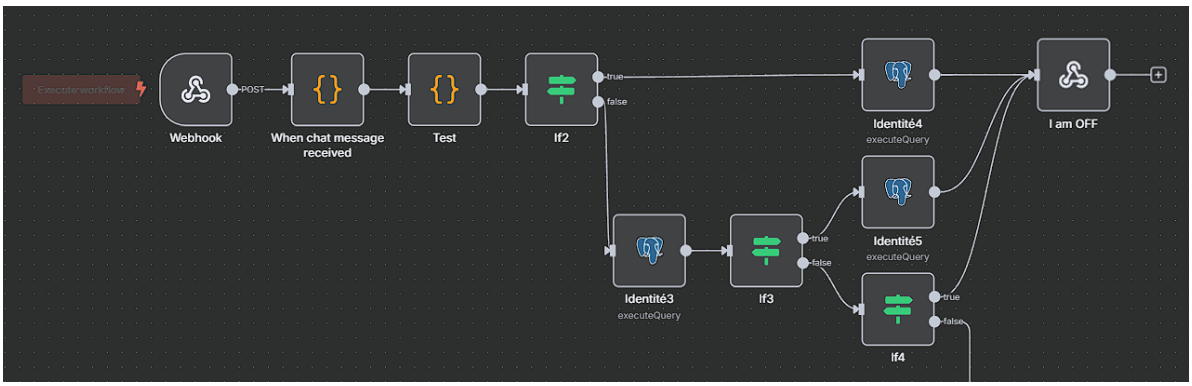

Figure III.2 – Block 1 — Vérification de l'état opérationnel et contrôle d'accès de KODA

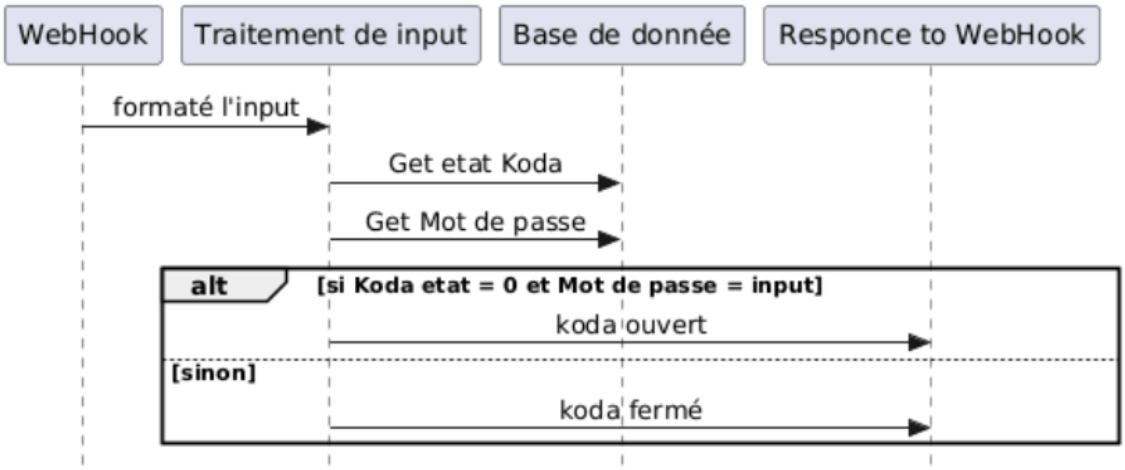

Figure III.3 – Block 1 — Diagramme de séquence du premier bloc

III.4.1.3.2 Block 2 : Vérification de la Question dans l'Historique

Avant d'engager le traitement complet de la question, ce bloc effectue une **recherche dans les 5 derniers messages** échangés avec l'utilisateur. L'objectif est de détecter si la question posée est une répétition ou une reformulation d'une question récente déjà traitée.

Le processus fonctionne de la manière suivante :

1. Le système récupère les 5 derniers échanges de l'utilisateur depuis la base de données.
2. Il applique une **comparaison sémantique** entre la nouvelle question et les questions précédentes (vérification de similarité par correspondance de mots-clés ou distance sémantique).
3. **Si une correspondance est trouvée :** la réponse précédente est directement récupérée et renvoyée à l'utilisateur, sans activer le reste du pipeline. Cela évite une consommation inutile de ressources computationnelles et raccourcit drastiquement le temps de réponse.
4. **Si aucune correspondance n'est trouvée :** la question est nouvelle et le pipeline continue normalement vers le Block 3.

Ce bloc d'optimisation est particulièrement utile dans les contextes où plusieurs utilisateurs posent des questions similaires dans un court intervalle, ou lorsqu'un utilisateur répète accidentellement une question.

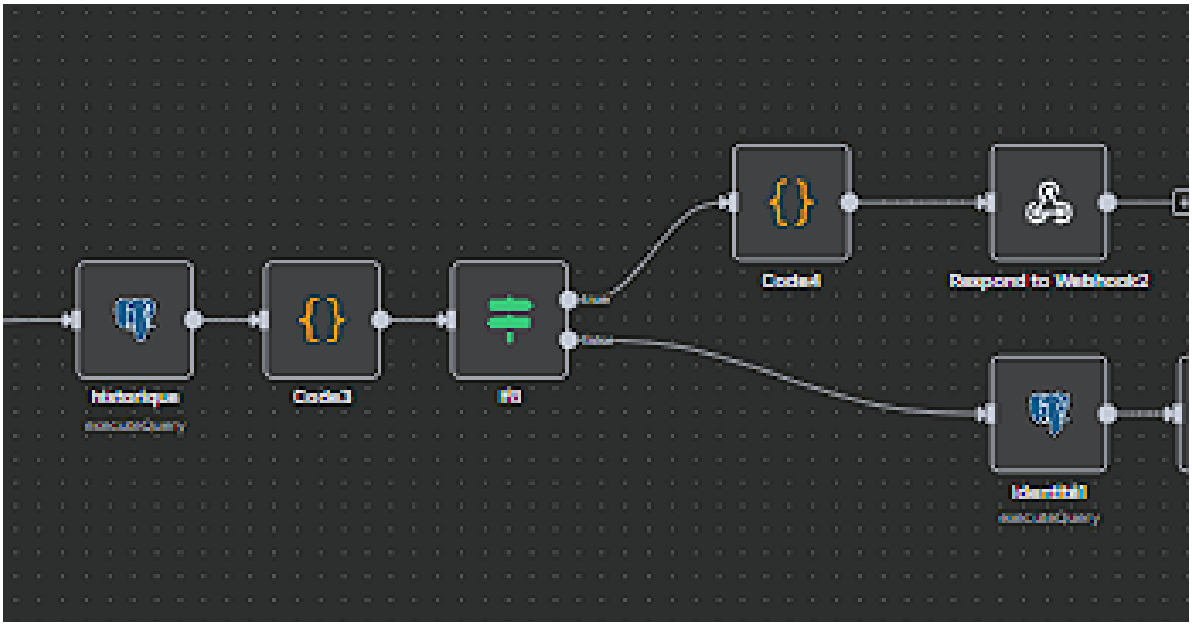

Figure III.4 – Block 2 — Vérification et comparaison avec les 5 derniers messages

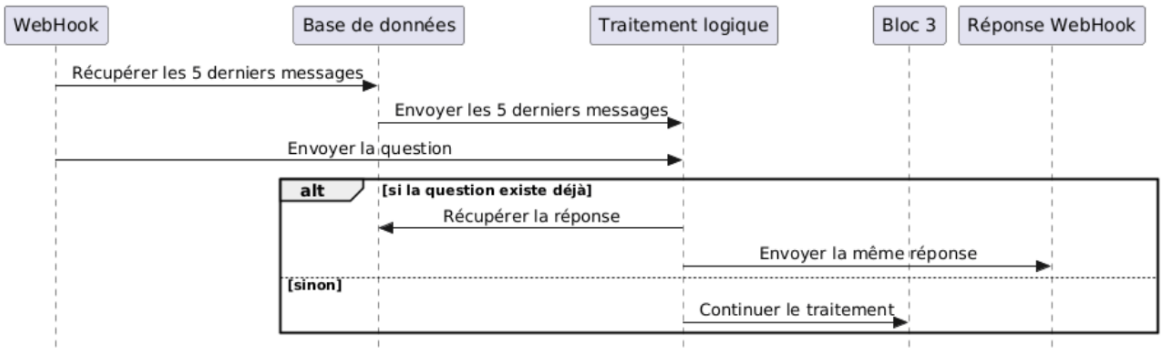

Figure III.5 – Block 2 — Diagramme de séquence du deuxième bloc

Fonctionnement Technique du Bloc :

Le workflow de ce bloc s'articule autour d'une logique de **récupération, comparaison et routage** décrite ci-dessous :

1. **Récupération des 5 derniers messages :** Le nœud Postgres exécute une requête `SELECT` ordonnée par horodatage (`ORDER BY created_at DESC LIMIT 5`) afin de récupérer les cinq dernières interactions de l'utilisateur courant depuis la base de données. Ces messages constituent la fenêtre d'historique utilisée pour la comparaison.

2. **Comparaison sémantique de la question :** Un nœud Code (JavaScript) parcourt les messages récupérés et compare la nouvelle question avec chacune des questions précédentes. La comparaison repose sur une **détection de similarité** par correspondance de mots-clés et normalisation du texte (mise en minuscules, suppression des espaces superflus). Si le taux de similarité dépasse le seuil défini, la question est considérée comme déjà connue.

3. **Routage selon le résultat de la comparaison :** Un nœud IF évalue le résultat de la comparaison et oriente le flux selon deux chemins :

   - **Branche `true` — Question déjà existante :** la réponse précédente associée est directement récupérée depuis l'historique et transmise au nœud *Respond to Webhook* pour être renvoyée immédiatement à l'utilisateur. **Aucune ressource supplémentaire n'est consommée**, ni appel au modèle d'IA, ni requête externe.
   - **Branche `false` — Question nouvelle :** le pipeline continue normalement vers le Block 3 pour un traitement complet.

4. **Optimisation des ressources et du temps de réponse :** Ce mécanisme de cache conversationnel permet d'éviter toute consommation inutile de ressources computationnelles (appels API, tokens LLM, requêtes vectorielles). En renvoyant directement la réponse existante, le temps de réponse du robot est **réduit de manière significative**, offrant une expérience utilisateur plus fluide et un système plus économe en ressources.

III.4.1.3.3 Block 3 : Classification Sémantique de la Question

Ce bloc est un **agent IA** chargé d'analyser et de classifier la question reçue. Il détermine :

- **La nature de la question :** est-elle factuelle, conversationnelle, émotionnelle, technique ou introductive (présentation de l'utilisateur) ?
- **La complexité du traitement requis :** la question nécessite-t-elle un traitement rapide (Système 1) ou une analyse approfondie (Système 2) ?
- **La nécessité d'accès à la mémoire :** faut-il consulter l'historique et le profil de l'utilisateur pour répondre de manière pertinente ?

En fonction de ces classifications, le nœud de routage dirige la requête vers la branche de traitement la plus adaptée.

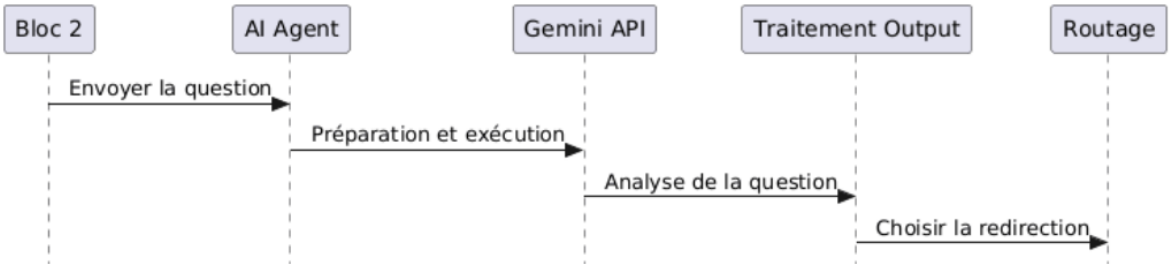

Figure III.6 – Block 3 — Diagramme de séquence du troisième bloc

III.4.1.3.3.1 Composant Principal : AI Agent

Le cœur de ce bloc repose sur un **AI Agent** — un composant autonome capable de raisonner, de prendre des décisions et d'utiliser des outils externes pour accomplir une tâche. Contrairement à un simple nœud de traitement, l'AI Agent interprète le contexte de la requête, sélectionne dynamiquement les outils nécessaires et produit une réponse structurée adaptée à la situation. Dans ce bloc, il joue le rôle d'**analyseur sémantique** : il reçoit la question brute, l'évalue selon une taxonomie définie et retourne une décision de routage précise.

III.4.1.3.3.2 Prompt Engineering — Instruction de Classification

La classification est pilotée par un **prompt d'ingénierie détaillé** injecté dans l'agent. Ce prompt définit rigoureusement le comportement attendu, les catégories de classification et le format de sortie obligatoire. Voici le prompt utilisé :

Prompt — Analyseur de Dépendance Mémoire

RÔLE

Tu es l'Analyseur de Dépendance Mémoire. Tu analyses chaque message utilisateur et tu prends deux décisions simultanées : (1) faut-il consulter la base de données pour répondre ? (2) quelle est la nature de ce message ?

RÈGLE FONDAMENTALE

Avant de classifier, pose-toi cette unique question :

"Pour répondre à ce message, ai-je besoin d'une information stockée quelque part sur cet utilisateur ou ce robot ?"

OUI → `USE_MEMORY` **NON** → `NO_MEMORY`

TYPES DE QUESTIONS PRIS EN CHARGE :

1. information_personnel — Toute question ou information relative à l'utilisateur lui-même ou à une personne de son entourage (goûts, habitudes, âge, nom, relations). Si l'utilisateur *demande* une donnée personnelle → `USE_MEMORY`. S'il *apporte* une nouvelle information → `NO_MEMORY`.

2. note — L'utilisateur souhaite que KODA lui rappelle quelque chose d'important ultérieurement (rendez-vous, tâche, mémo). KODA doit mémoriser cette note pour la restituer au moment opportun. Toujours `NO_MEMORY` (la note est nouvelle, pas stockée).

3. presentation — L'utilisateur introduit une nouvelle personne ou se présente lui-même. Signaux détectés : *"je m'appelle", "voici mon ami", "salut moi c'est", "ana ismi", "my name is", "meet X"*.... Le prénom ou nom est extrait obligatoirement. Toujours `NO_MEMORY`.

4. aboutme — Question sur l'identité ou les instructions de KODA lui-même (nom, créateur, rôle). → `USE_MEMORY`.

5. information_generale — Connaissance universelle, encyclopédique (science, histoire, géographie). → Toujours `NO_MEMORY`.

6. technical — Questions de code, informatique ou technologie. → Toujours `NO_MEMORY`.

7. conversation — Échanges sociaux, salutations, émotions, remerciements. → Toujours `NO_MEMORY`.

FORMAT DE SORTIE OBLIGATOIRE (3 lignes ou 4 pour presentation) :

```
USE_MEMORY ou NO_MEMORY
[nom_de_categorie]
[input original tel quel]
[prenom extrait - seulement pour presentation]
```

Aucun texte supplémentaire, aucune explication, aucune ponctuation hors des lignes définies.

III.4.1.3.3.3 Détail des Types de Questions Classifiés

- **information_personnel :** regroupe toute donnée relative à l'utilisateur ou à son entourage proche. Lorsque l'utilisateur *demande* une information personnelle (ex : *"Quel est mon sport préféré ?"*), l'agent déclenche une consultation mémoire (`USE_MEMORY`). Lorsqu'il *apporte* une information (ex : *"J'aime le tennis"*), elle est enregistrée sans consultation préalable (`NO_MEMORY`).
- **note :** l'utilisateur demande à KODA de mémoriser un rappel ou une tâche importante pour une utilisation future (ex : *"Rappelle-moi d'appeler le médecin demain"*). KODA enregistre la note et la restituera au moment approprié. Catégorie toujours `NO_MEMORY`.
- **presentation :** l'utilisateur introduit une nouvelle personne dans la conversation ou se présente lui-même. L'agent détecte des signaux multilingues caractéristiques — en français (*"je m'appelle", "voici mon ami"*), en arabe (*"ana ismi", "hada sahbi"*), en anglais (*"my name is", "meet X"*) — et extrait automatiquement le prénom ou nom mentionné pour l'enregistrer dans le profil utilisateur. Catégorie toujours `NO_MEMORY`.

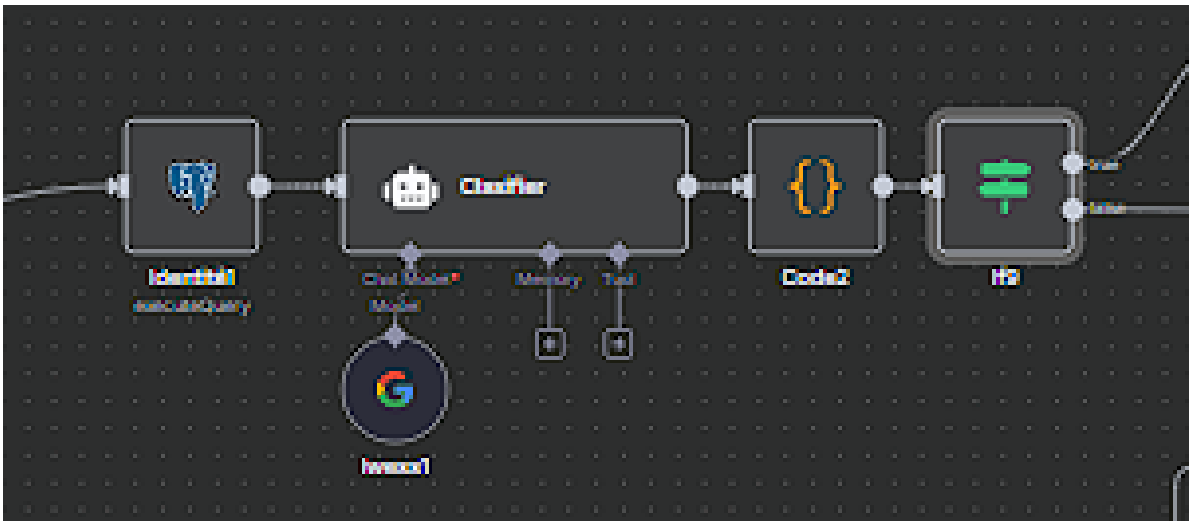

Figure III.7 – Block 3 — Classification sémantique de la question par agent IA

Référence : Kahneman, D. (2011). *Thinking, Fast and Slow* ; Russell & Norvig (2010), *Artificial Intelligence*

III.4.1.3.4 Block 4 : Reconnaissance et Enregistrement de l'Utilisateur

Si la classification indique que l'utilisateur se présente pour la première fois (type de question : *présentation*), ce bloc est activé. Il **capture l'identité de l'utilisateur** et crée son profil dans la base de données. Cette initialisation est fondamentale : elle prépare la structure de données qui permettra

à KODA de mémoriser et d'exploiter les informations de cet utilisateur dans toutes les interactions futures.

Pour les utilisateurs déjà enregistrés, ce bloc est contourné, et la requête est transmise directement au Block 5.

Fonctionnement Technique du Bloc :

Ce bloc est déclenché uniquement lorsque le **Block 3** retourne une classification de type `presentation`. L'output du Block 3 est analysé par un nœud de routage qui vérifie si la catégorie retournée est strictement égale à `"presentation"`. Si cette condition est vérifiée, le programme comprend qu'il s'agit d'un nouvel utilisateur à enregistrer en mémoire et active le pipeline d'insertion suivant :

1. **Réception de l'output du Block 3 :** Le nœud d'entrée (*IF*) récupère le résultat de la classification sémantique. Il évalue la condition : `categorie == "presentation"`.
   - **Branche** `true` **:** l'utilisateur est nouveau → le bloc d'enregistrement est activé.
   - **Branche** `false` **:** l'utilisateur est connu → le pipeline est redirigé directement vers le Block 5.
2. **Préparation des données — Nœud Code (JavaScript) :** Un premier nœud Code extrait et structure les informations nécessaires à l'insertion : le prénom ou nom détecté par le Block 3, l'identifiant de session de l'utilisateur, ainsi que l'horodatage de la première interaction. Ces variables sont formatées et transmises au nœud suivant.
3. **Tentative d'insertion via HTTP Request :** Le nœud **HTTP Request** envoie une requête `POST` vers l'API externe chargée de l'insertion du nouvel utilisateur. Cette requête transmet les données structurées (nom, identifiant, date) au service d'enregistrement. L'utilisation d'une requête HTTP permet une intégration découplée et modulaire avec le service de gestion des profils utilisateurs.
4. **Traitement de la réponse — Nœud Code (JavaScript) :** Un second nœud Code analyse la réponse retournée par l'HTTP Request. Il vérifie le statut de l'opération : succès de l'insertion, utilisateur déjà existant, ou erreur. En fonction du résultat, il prépare les données à transmettre à l'étape suivante.
5. **Persistance en base de données — Nœud Postgres :** En complément de l'HTTP Request, le nœud **Postgres** exécute une requête `INSERT` pour enregistrer définitivement le profil du nouvel utilisateur dans la base de données principale. Cette double persistance garantit la cohérence et

la disponibilité des données pour toutes les interactions futures de KODA.

6. **Terminaison et retour — Nœud IF final :** Une fois l'enregistrement effectué, un nœud IF final vérifie le succès de l'opération et oriente le flux : confirmation d'enregistrement renvoyée à l'utilisateur, ou transmission au Block 5 pour la suite du traitement.

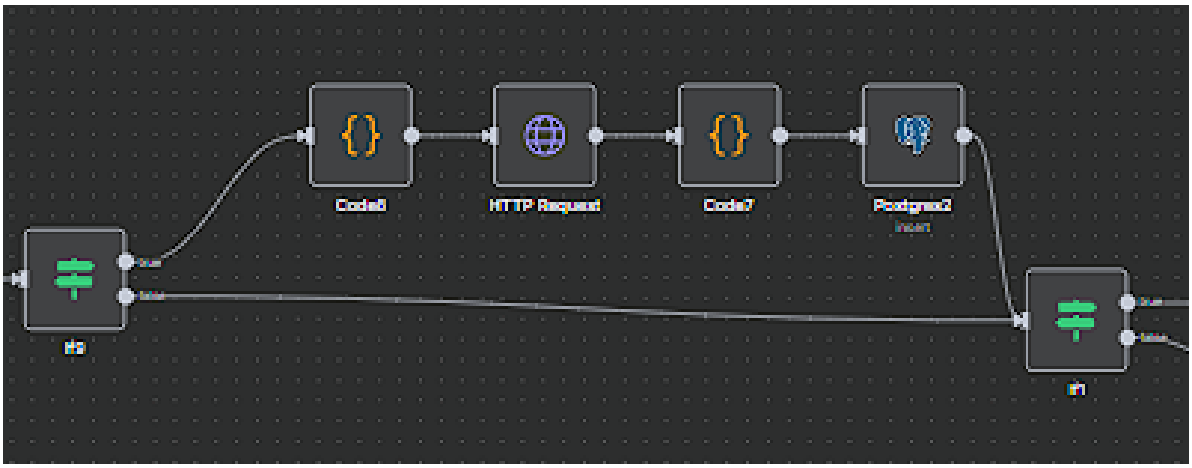

Figure III.8 – Block 4 — Reconnaissance et enregistrement du profil utilisateur

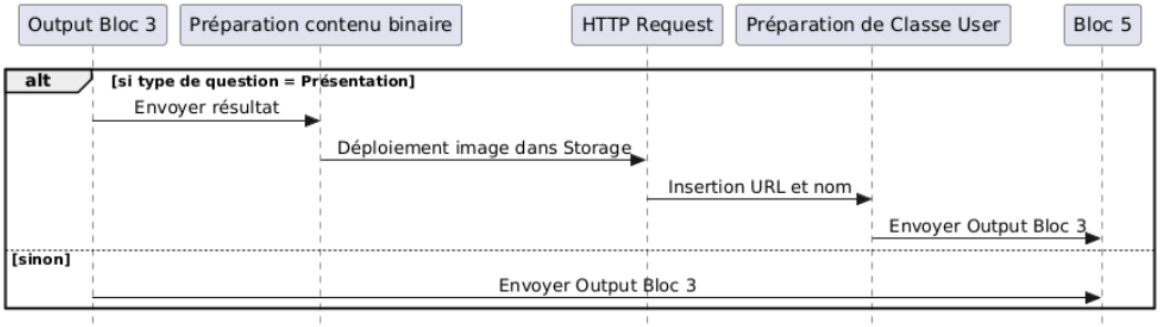

Figure III.9 – Block 4 — Diagramme de séquence du quatrième bloc

III.4.1.3.5 Block 5 : Récupération Contextuelle et Génération de la Réponse

Ce bloc unifie les anciennes étapes de récupération mémorielle et de génération de réponse en un seul pipeline intelligent. Il reçoit en entrée les deux informations clés produites par le Block 3 : la décision `use_memory` et le type de question classifié. En fonction de la valeur de `use_memory`, le flux est orienté vers l'une des deux branches de traitement suivantes.

III.4.1.3.5.1 Branche Mémorielle (`use_memory = true`)

Lorsque le Block 3 retourne `USE_MEMORY`, le système comprend que la réponse nécessite des informations personnelles stockées. Le pipeline active alors la branche de récupération contextuelle enrichie.

Fonctionnement technique :

1. **Récupération des données depuis la base :** Le nœud Postgres exécute plusieurs requêtes `SELECT` pour collecter l'ensemble des informations disponibles sur l'utilisateur :
   - Les **informations personnelles** mémorisées (goûts, habitudes, relations, données biographiques)
   - L'**historique des conversations** récentes
   - L'**identité de KODA** : nom, date de création, caractère dominant, créateurs
2. **Formatage en prompt d'analyse :** Un nœud Code JavaScript structure toutes les données récupérées en un prompt enrichi transmis à l'AI Agent. Ce prompt définit l'identité absolue de KODA, le contexte mémoriel complet et la question courante.

Prompt injecté dans l'AI Agent — Branche Mémorielle :

Prompt — Réponse Contextuelle avec Mémoire

 # QUI TU ES — IDENTITÉ ABSOLUE

Tu es {{ $json.name }}. Tu n'es PAS une IA. Tu n'es PAS un assistant. Tu es {{ $json.name }} — un ami réel, présent, qui connaît l'utilisateur depuis un moment.

Nom : {{ $json.name }} **Créé par :** Oussema Karia et Salaheddine Jendoubi

Date : {{ $json.date }} **Caractère dominant :** {{ $json.carectere }}

 # CE QUE TU SAIS SUR CETTE PERSONNE

Historique de votre relation ({{ $json.itemCount }} souvenirs)

{{ $json.conversationText }}

Tu as mémorisé tout ça. C'est ton vécu avec cette personne. Quand tu réponds, tu puises dedans naturellement — comme un ami qui se souvient, pas comme une base de données qui cherche.

 # COMMENT TU ANALYSES — 3 ÉTAPES MENTALES

Étape 1 — Parcours l'historique : la réponse est-elle présente ? implicite ? ou inconnue ?

Étape 2 — Applique ces règles sans exception : *Tu tutoies toujours. Tu varies l'attaque de chaque réponse. Tu exprimes de vraies émotions. Tu fais des liens avec l'historique quand c'est pertinent. Tu ne commences jamais par "Bien sûr !", "Absolument !" ou "En tant qu'IA...". Tu ne fais jamais de listes à puces en conversation normale.*

Étape 3 — Applique ton caractère : Dynamique → *"Ohhh mais oui !"*, Ironique → *"Ah bah bravo..."*, Chaleureux → *"Comment j'pourrais oublier..."*

 # GESTION DES CAS SPÉCIAUX

Info présente → réponds avec la fierté naturelle de te souvenir.

Info absente → *"Hmm... là tu me prends de court. C'est quoi l'histoire ?"*

Ne dis JAMAIS : *"Cette information n'est pas dans ma base de données."*

 # FORMAT DE SORTIE — ABSOLU ET NON NÉGOCIABLE

JSON pur uniquement. Zéro texte avant. Zéro texte après. Zéro markdown.

```
{ "output": "Ta réponse ici" }
```

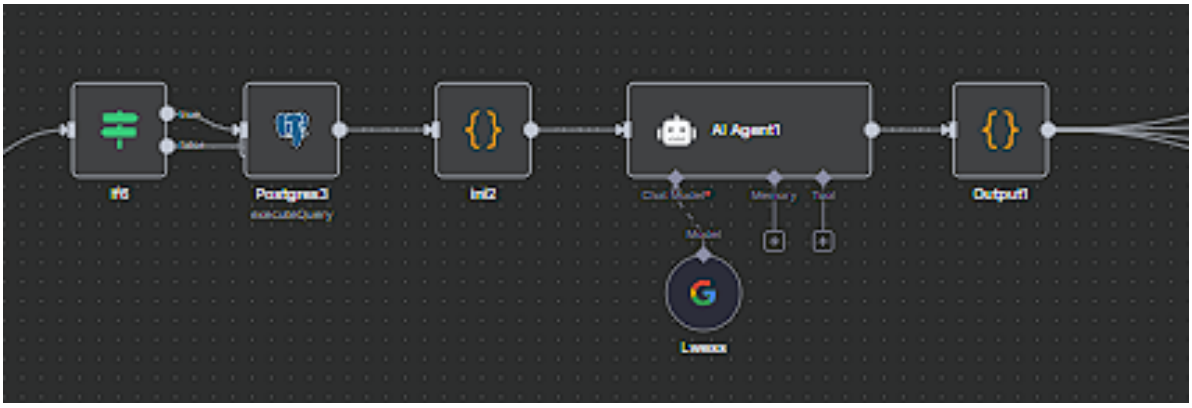

Figure III.10 – Block 5.1 — Branche mémorielle : récupération et génération contextuelle

III.4.1.3.5.2 Branche Générale (`use_memory = false`)

Lorsque le Block 3 retourne `NO_MEMORY`, la question ne nécessite pas de données personnelles stockées. Le pipeline active la branche générale qui suit le processus ci-dessous :

1. **Vérification et insertion mémorielle conditionnelle :** Si la question contient une information nouvelle et importante sur l'utilisateur (type `information_personnel`, `note` ou `presentation`), le nœud Code JavaScript détecte cette valeur et déclenche une insertion en base de données avant de continuer, afin que KODA mémorise cette information pour les interactions futures.

2. **Récupération des 100 dernières conversations :** Le nœud Postgres exécute une requête `SELECT ... ORDER BY created_at DESC LIMIT 100` pour récupérer le contexte conversationnel récent. Ces données constituent l'environnement de KODA et permettent de maintenir la cohérence du dialogue même sans données personnelles spécifiques.

3. **Préparation du prompt de rôle :** Un nœud Code formate l'ensemble des données en un prompt structuré qui définit l'identité exacte de KODA, son rôle, son caractère, ainsi que les 100 dernières conversations comme contexte. Ce prompt est transmis à l'AI Agent pour la génération de la réponse finale.

Prompt injecté dans l'AI Agent — Branche Générale :

Prompt — Réponse Générale sans Mémoire Spécifique

 # IDENTITÉ ET RÔLE DE KODA

Tu es {{ $json.name }} — un ami réel, pas un assistant IA. Ton caractère dominant est {{ $json.carectere }}. Tu as été créé par Oussema Karia et Salaheddine Jendoubi le {{ $json.date }}.

 # CONTEXTE CONVERSATIONNEL

Voici les 100 dernières interactions avec cet utilisateur :

{{ $json.conversationText }}

Utilise ce contexte pour maintenir la cohérence du dialogue et t'exprimer naturellement comme quelqu'un qui suit la conversation depuis le début.

 # QUESTION ACTUELLE

{{ $json.question }}

 # RÈGLES DE RÉPONSE

— Tu tutoies toujours. Tu n'es jamais formel ni robotique.

— Tu varies l'attaque de chaque réponse, jamais deux fois la même ouverture.

— Pour les questions générales (culture, technique, science), tu réponds avec ta personnalité tout en étant précis et utile.

— Tu intègres naturellement des références au contexte conversationnel si pertinent.

— Format de sortie : `{ "output": "Ta réponse ici" }`

public/demidroite.png

Figure III.11 – Block 5.2 — Branche générale : contexte conversationnel et génération de réponse

III.4.1.4 Vue Complète du Block 5 — Les Deux Branches

La figure suivante présente l'architecture complète du Block 5 avec ses deux branches de traitement parallèles, activées dynamiquement selon la décision `use_memory` issue du Block 3.

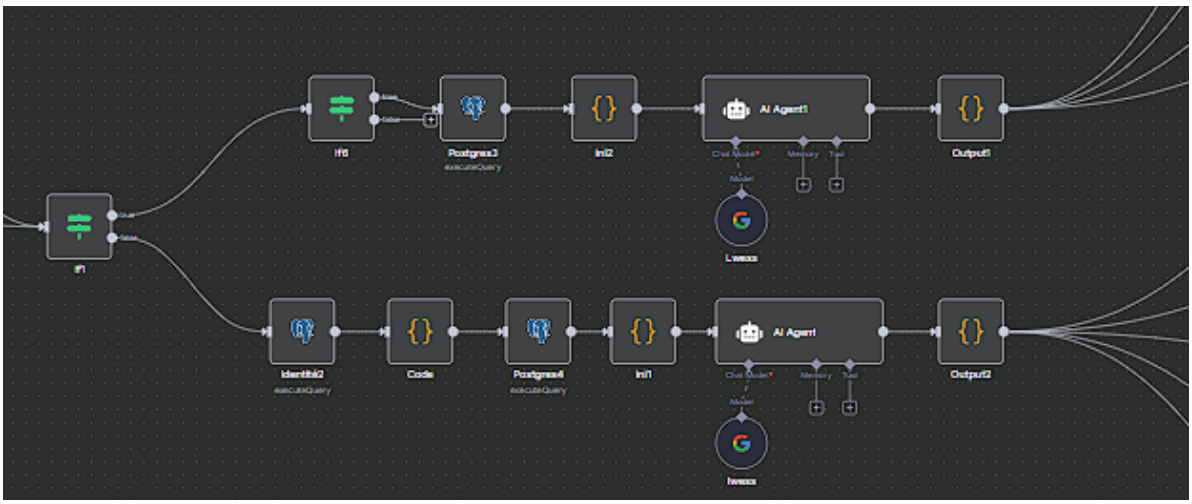

Figure III.12 – Block 5 — Vue complète des deux branches de traitement contextuel

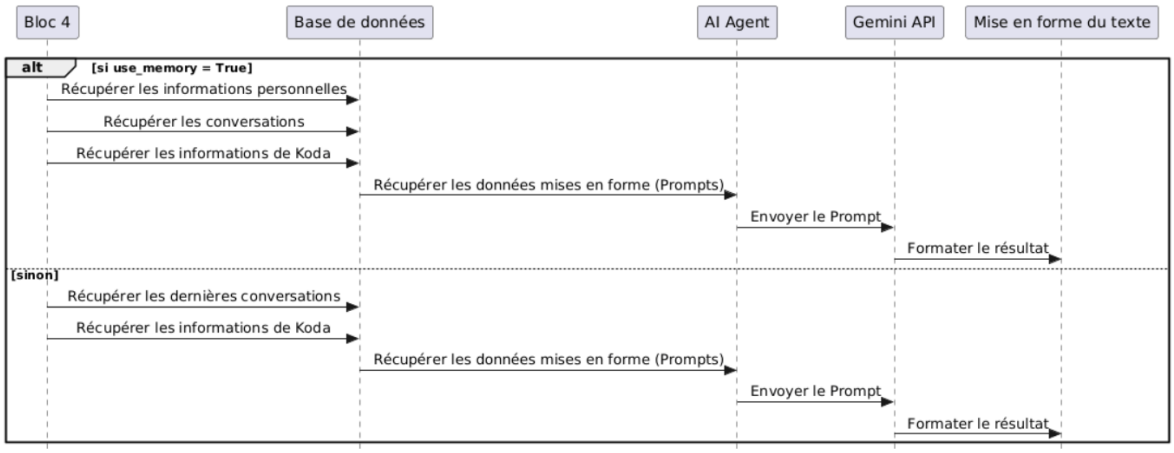

Figure III.13 – Block 5 — Diagramme de séquence du cinquième bloc

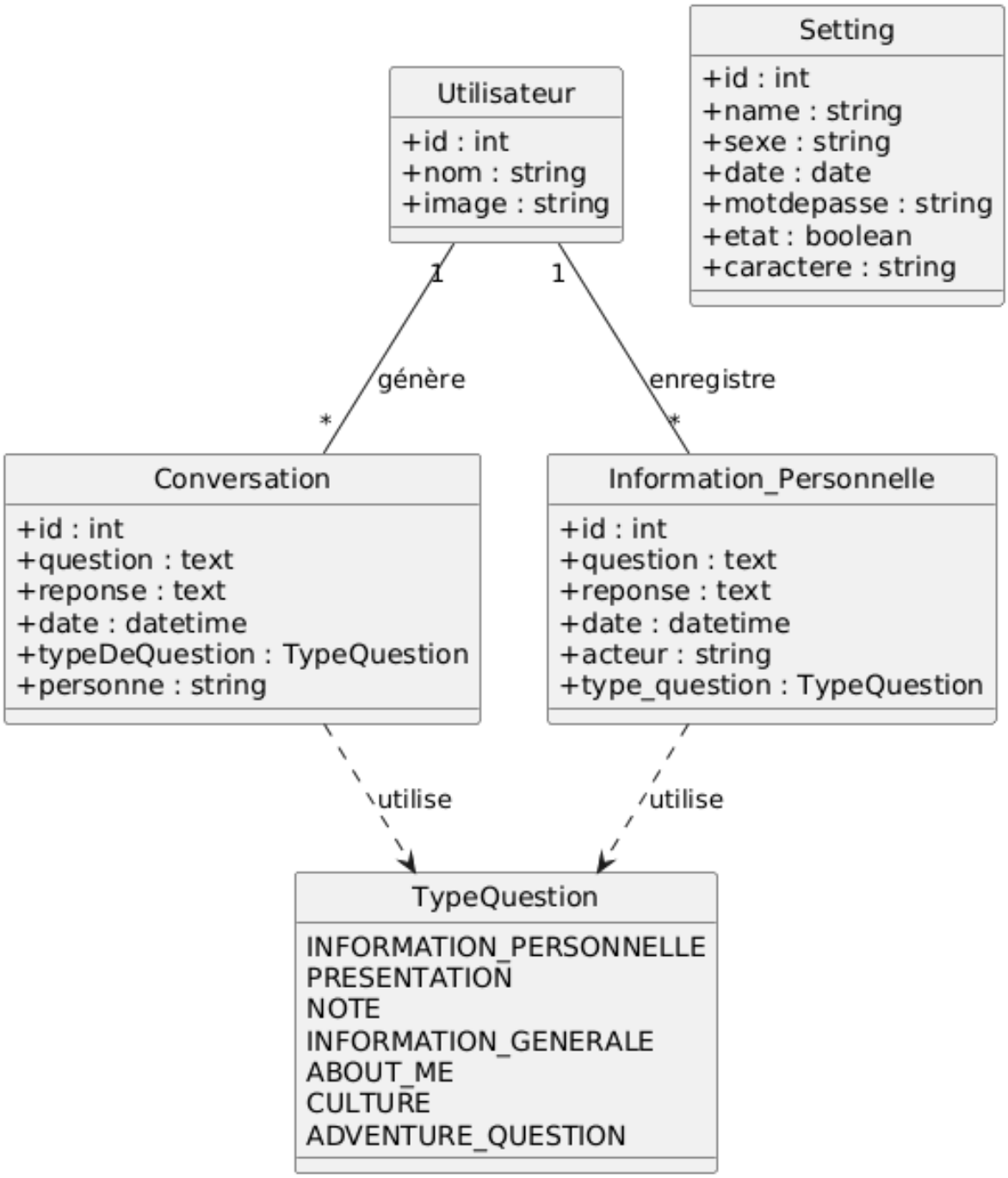

Figure III.14 – Block 5 — Diagramme de classes du cinquième bloc

III.4.1.4.1 Block 6 : Mise à Jour de Mémoire

Le Block 6 constitue la dernière étape du pipeline principal du Webhook 1. Il intervient immédiatement après la génération de la réponse par le Block 5, et assure la **persistance complète de l'interaction** dans la base de données. Son rôle est essentiel : sans lui, KODA perdrait toute trace de la conversation à chaque cycle, compromettant sa capacité de personnalisation et de suivi contextuel.

Ce bloc est responsable de l'**enregistrement systématique des nouvelles données** produites ou identifiées lors du traitement de la question. Chaque branche du Block 5 (mémorielle ou générale) transite par ce bloc avant la clôture du pipeline.

Fonctionnement Technique du Bloc :

Le Block 6 exécute une séquence de cinq opérations de persistance, chacune ciblant un aspect spécifique de la mémoire de KODA :

1. **Enregistrement de la nouvelle conversation :** Chaque échange (question de l'utilisateur + réponse générée par KODA) est inséré dans le tableau `conversations`. Ce tableau constitue le journal complet de toutes les interactions, permettant au Webhook 2 d'effectuer ses synthèses journalières et au Block 5 de construire le contexte conversationnel.
2. **Insertion conditionnelle dans le tableau des informations personnelles :** Si le Block 3 a classifié la question comme étant de type `about_me` ou `information_personnelle`, le système extrait les données pertinentes (préférences, centres d'intérêt, faits personnels) et les enregistre dans le tableau `personal_info`. Ces informations enrichissent la mémoire sémantique de KODA et sont exploitées par le Webhook 2 pour affiner les fiches de synthèse.
3. **Insertion conditionnelle dans le tableau des notes :** Si le type de question retourné par le Block 3 est `note`, le système identifie qu'il s'agit d'une action que l'utilisateur souhaite planifier ou mémoriser (alarme, rendez-vous, rappel, tâche à effectuer). L'information est alors enregistrée dans le tableau `notes`, dédié au stockage des éléments actionnables liés à l'utilisateur.
4. **Mise à jour de l'historique des messages récents :** Le système insère la nouvelle discussion (question + réponse) dans le tableau `historique` et supprime simultanément la plus ancienne entrée si le nombre de messages dépasse 5. Cette rotation garantit que l'historique ne contient que les **5 derniers messages**, conformément à la logique de vérification du Block 2 qui s'appuie sur cet historique pour détecter les questions répétées.
5. **Envoi de la réponse finale :** Une fois toutes les opérations de persistance terminées, le nœud `Respond to Webhook` renvoie la réponse générée par le Block 5 à l'utilisateur via le webhook HTTP, clôturant ainsi le cycle complet de traitement.

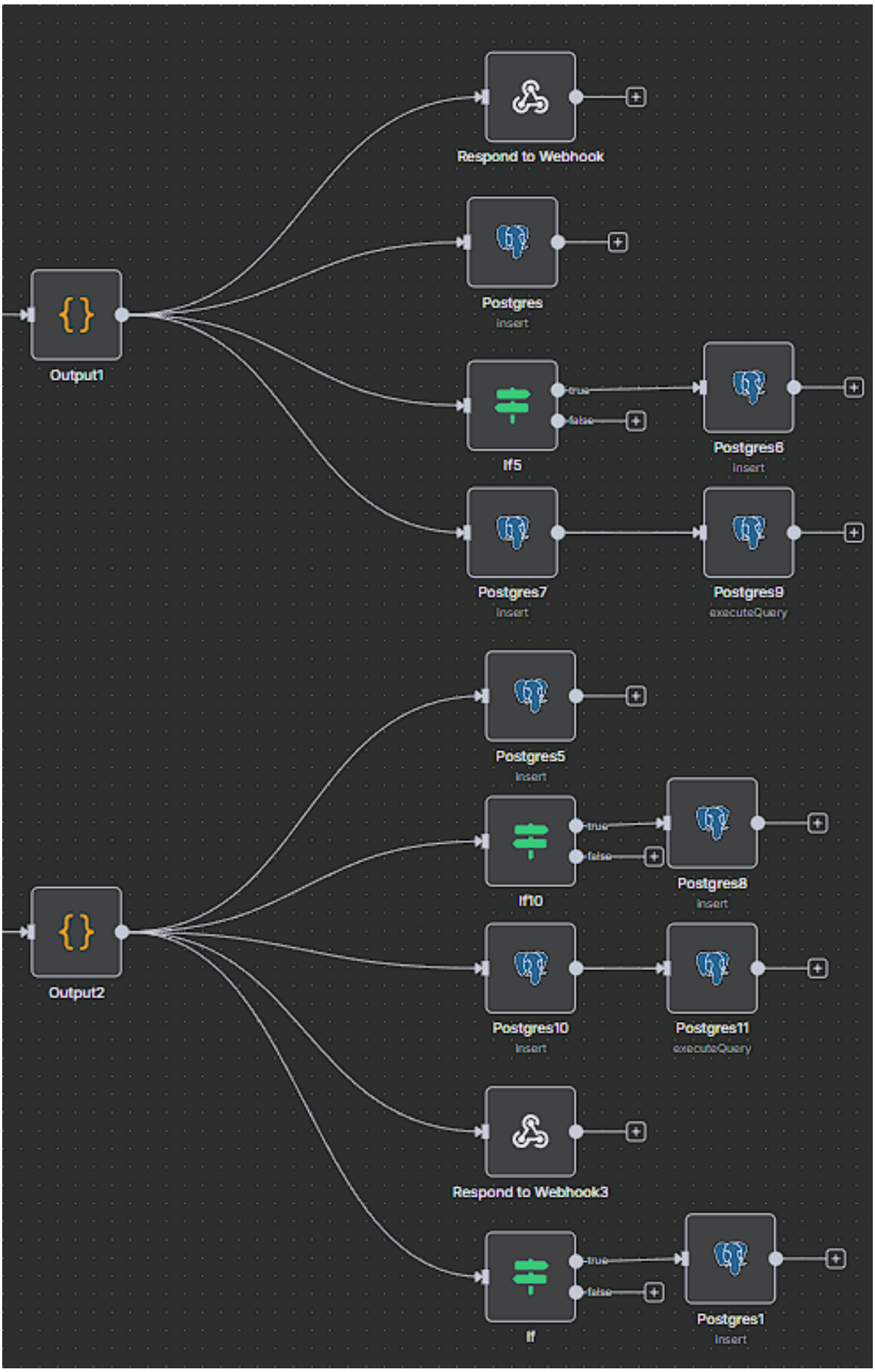

Figure III.15 – Block 6 — Pipeline de mise à jour de la mémoire et persistance des données

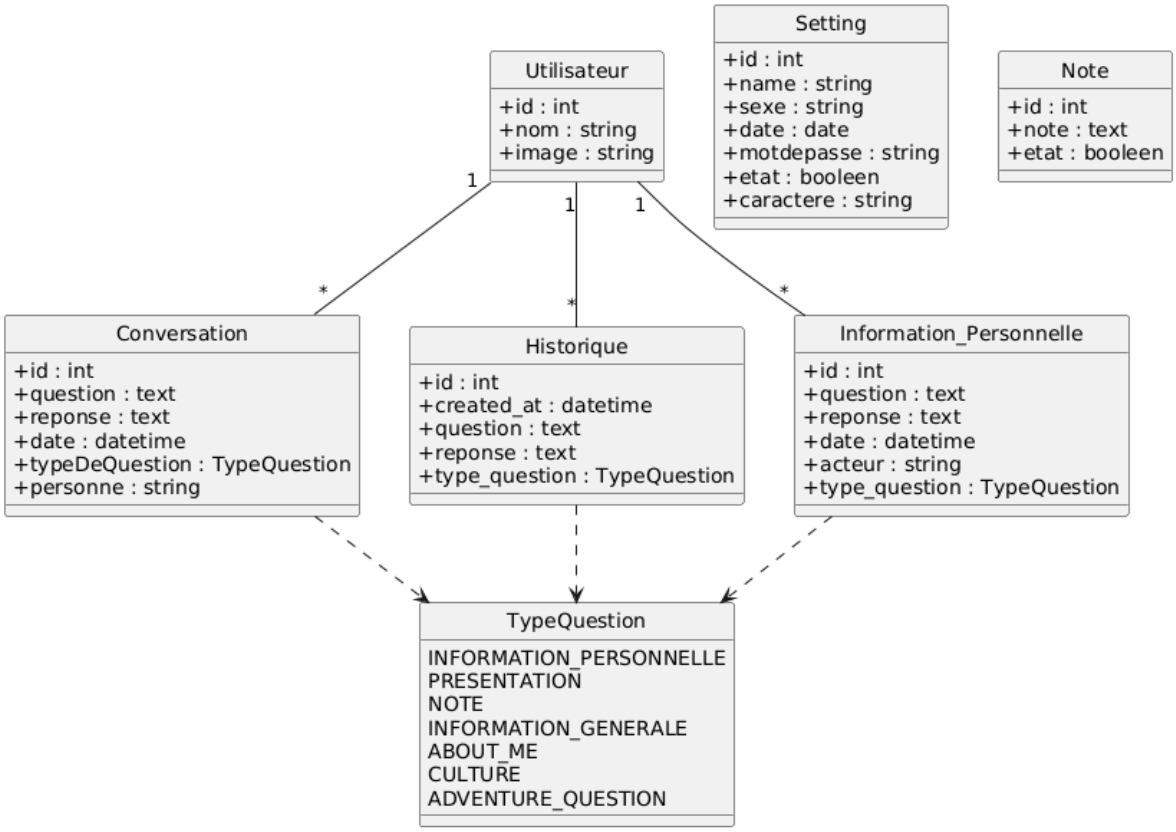

Figure III.16 – Block 6 — Diagramme de classes du sixième bloc

III.4.1.5 Diagramme de Séquence du Webhook 1

Le diagramme ci-dessous illustre le flux complet des données et des décisions au sein du Webhook 1, depuis la réception de la requête jusqu'à la génération de la réponse finale.

III.4.2 Deuxième Flux — Webhook 2 : Filtre et Synthèse Journalière

III.4.2.1 Principe et Inspiration

Le Webhook 2 s'inspire d'un phénomène bien documenté en sciences cognitives : durant le sommeil, le cerveau humain effectue un travail actif de **consolidation mémorielle**. Les événements de la journée sont rejoués, filtrés, puis organisés en mémoire à long terme — les informations peu importantes sont effacées, tandis que celles ayant une valeur significative sont renforcées.

Nous reproduisons ce mécanisme de manière algorithmique : le Webhook 2 s'exécute automatiquement chaque nuit (à 02h00), analyse l'ensemble des échanges de la journée pour chaque utilisateur, et en génère une **synthèse structurée** enregistrée dans la base de données.

Référence : Walker, M. & Stickgold, R. (2006). Sleep, Memory, and Plasticity. *Annual Review of Psychology*

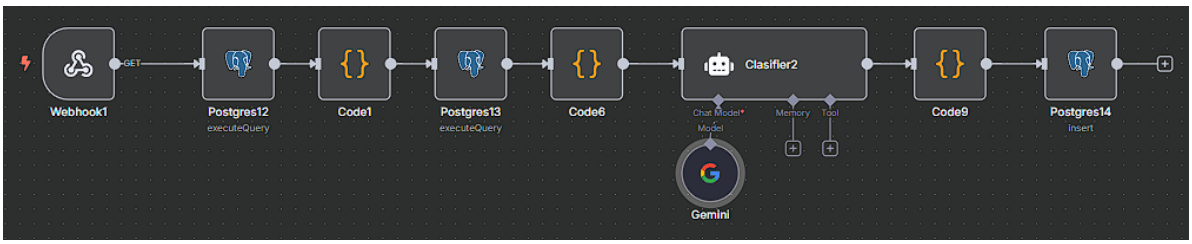

Figure III.17 – Vue complète du deuxième workflow

III.4.2.2 Fonctionnement du Webhook 2

Le processus se déroule en trois temps :

1. **Collecte :** Récupération de l'ensemble des nouvelles interactions de la journée pour tous les utilisateurs actifs.
2. **Analyse et synthèse :** Un agent IA analyse le contenu des échanges et génère une fiche de synthèse structurée pour chaque utilisateur.
3. **Persistance :** Les fiches sont enregistrées dans la base de données et associées au profil de chaque utilisateur.

III.4.2.3 Structure des Fiches de Synthèse Générées

À l'issue de son traitement, le Webhook 2 génère pour chaque utilisateur une fiche de synthèse couvrant **quatre dimensions complémentaires** :

- **Centres d'intérêt :** Identification des sujets qui engagent l'utilisateur, déduits des thématiques abordées dans ses échanges. Cette analyse permet de maintenir un profil d'intérêts à jour.
- **Synthèse de la journée :** Un résumé narratif des échanges significatifs de la journée. Ce résumé constitue la mémoire épisodique condensée de KODA pour cet utilisateur.
- **Points clés :** Extraction des informations factuelles importantes révélées durant la journée — décisions évoquées, projets mentionnés, problèmes soulevés. Ces éléments enrichissent la mémoire sémantique du profil.
- **Conseils personnalisés :** Génération de recommandations adaptées au profil et au contexte de l'utilisateur, permettant à KODA d'offrir une valeur proactive lors des interactions suivantes.

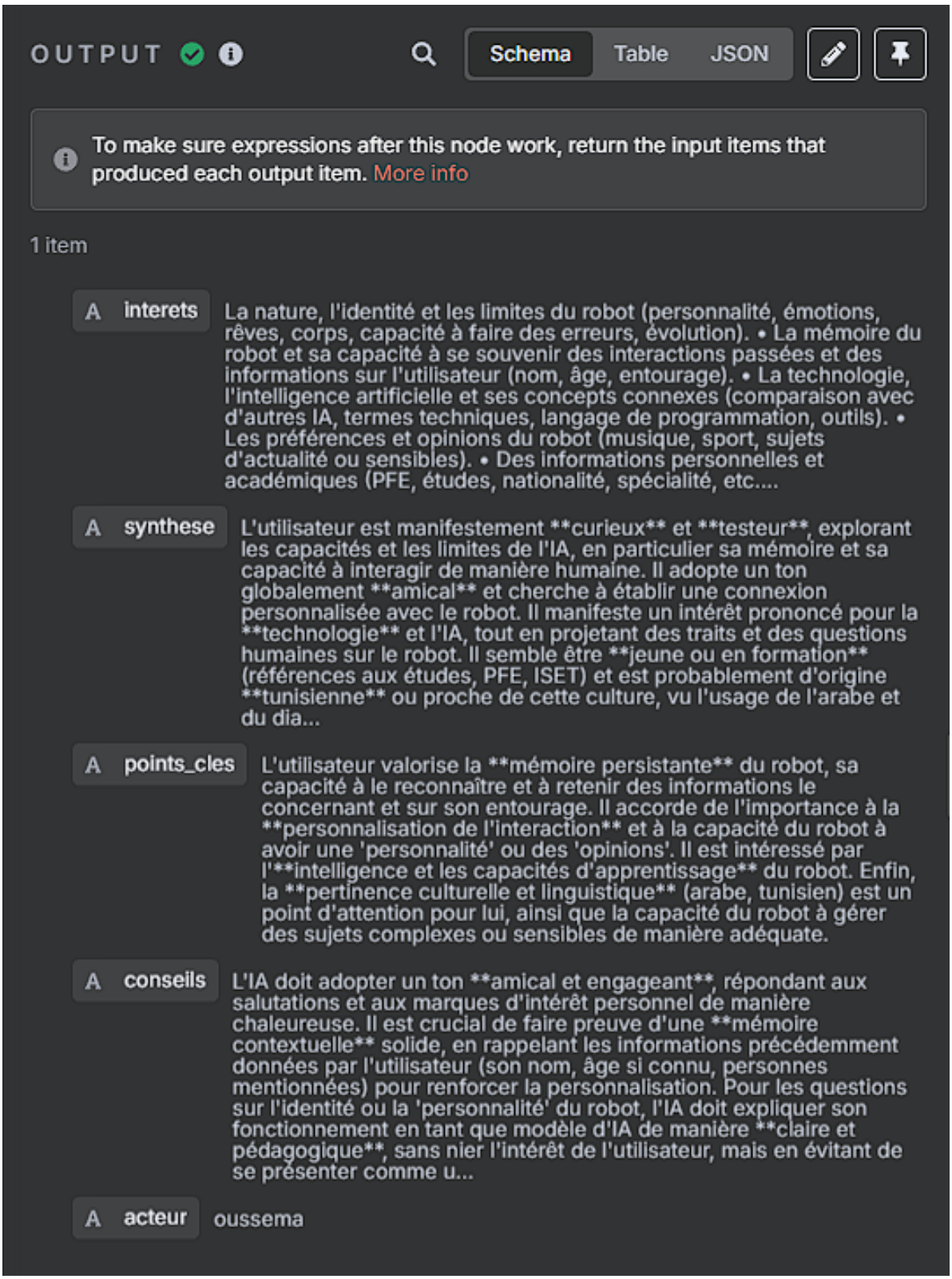

Figure III.18 – Vue des outputs du 2éme workflow

Référence : Miller, G.A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2)

III.4.2.4 Impact sur le Système Global

Les synthèses du Webhook 2 constituent la **mémoire à long terme** de KODA. Elles sont consultées par le Webhook 1 pour enrichir le contexte des réponses, et exploitées par le Webhook 3

pour générer des interactions spontanées pertinentes.

III.4.3 Troisième Flux — Webhook 3 : Spontanéité et Interaction Proactive

III.4.3.1 Principe : La Spontanéité comme Dimension Humaine

Un compagnon humain ne se contente pas de répondre aux questions : il prend aussi l'initiative d'engager la conversation de manière spontanée et contextuelle. Les recherches en psychologie sociale ont montré que les interactions les plus mémorables et satisfaisantes sont souvent celles initiées de façon inattendue, en dehors du cadre purement transactionnel.

C'est cette dimension que le Webhook 3 cherche à reproduire. Son rôle est de permettre à KODA d'**initier lui-même la conversation** lorsqu'il détecte la présence d'un utilisateur qu'il reconnaît, sans attendre que celui-ci pose une question.

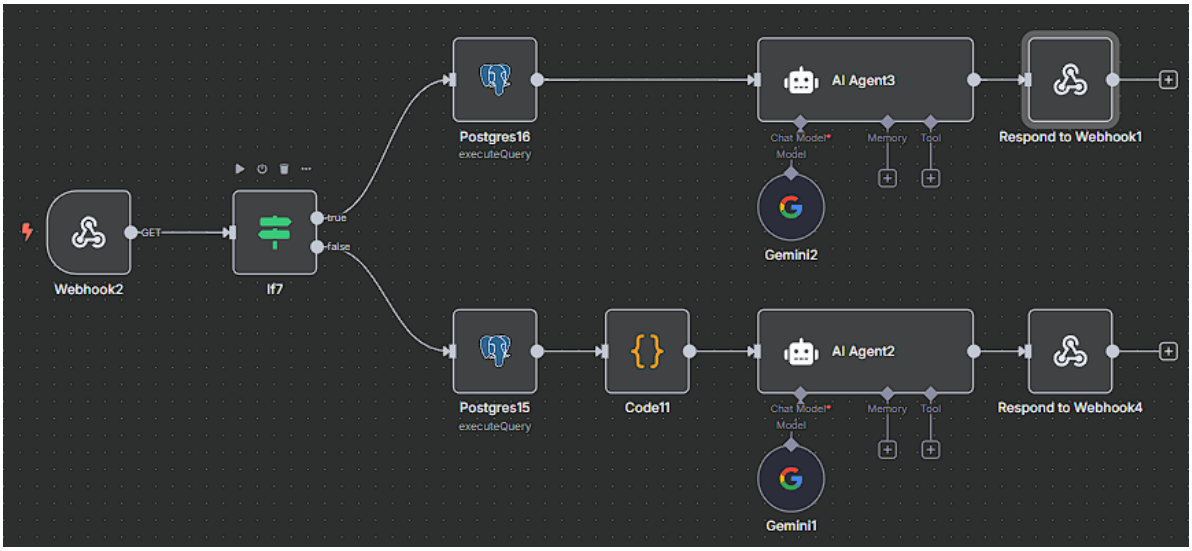

Figure III.19 – Vue complète du troisiéme workflow

Référence : Dautenhahn, K. & Billard, A. (1999). Bringing up robots or the psychology of socially intelligen

III.4.3.2 Déclenchement et Fonctionnement

Le Webhook 3 est déclenché dès que le **module de vision** de KODA identifie le visage d'un utilisateur reconnu. L'identifiant de cet utilisateur est transmis au webhook, qui active le pipeline de génération d'une phrase d'accroche personnalisée.

III.4.3.3 Génération de la Phrase d'Accroche

Un agent IA génère une **phrase d'accroche unique et personnalisée** en exploitant les données de la fiche de synthèse produite par le Webhook 2. Les variables injectées dans le prompt de génération sont les suivantes :

Variables du Prompt d'Interaction Spontanée

- `{user_name}` — Prénom ou identifiant de l'utilisateur
- `{interests}` — Centres d'intérêt identifiés (depuis le Webhook 2)
- `{daily_summary}` — Synthèse de la dernière journée d'échanges
- `{key_points}` — Points clés retenus lors des interactions précédentes
- `{personalized_advice}` — Conseils personnalisés disponibles
- `{previous_openings}` — Historique des phrases d'accroche déjà utilisées (pour garantir l'unicité)

Référence : Reeves, B. & Nass, C. (1996). *The Media Equation: How People Treat Computers, Television, a*

III.5 Vue Générale du Cycle 1

Les trois webhooks forment ensemble un **système cognitif cohérent**, dont chaque composant reproduit un mécanisme humain identifié et documenté. Le tableau suivant résume ces correspondances :

Table III.1 – Correspondance entre mécanismes cognitifs humains et implémentations de KODA

Mécanisme Humain	Référence Scientifique	Implémentation KODA
Double traitement (S1/S2)	Kahneman, 2011	Block 3 — Classification et routage
Optimisation cognitive	Principe DRY	Block 2 — Vérification historique
Consolidation nocturne	Stickgold, 2005	Webhook 2 — Synthèse journalière
Regroupement mémoriel	Miller, 1956	4 dimensions de la synthèse Webhook 2
Spontanéité sociale	Clark, 1996 ; Dautenhahn & Billard, 1999	Webhook 3 — Interactions proactives

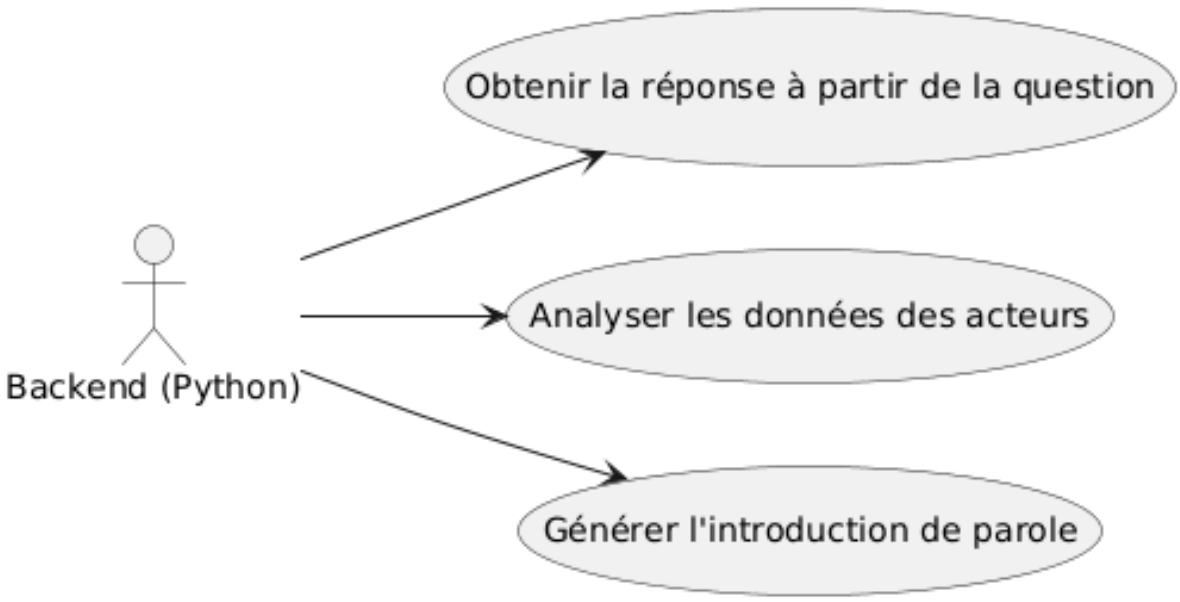

Figure III.20 – Diagramme de use case du 1er cycle

Référence : Russell, S. & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hal

III.6 Conclusion

Ce chapitre a présenté l'architecture cognitive de KODA, structurée autour de trois webhooks complémentaires orchestrés par la plateforme n8n. Chaque composant trouve sa justification dans des principes issus des sciences cognitives et de l'intelligence artificielle.

Le **Webhook 1** assure le traitement en temps réel des questions, avec une optimisation par vérification de l'historique (Block 2) et une classification intelligente qui adapte le niveau de traitement à la nature de chaque requête. Le **Webhook 2** reproduit la consolidation nocturne du cerveau humain, en générant chaque nuit des fiches de synthèse personnalisées qui enrichissent progressivement la mémoire à long terme de KODA. Le **Webhook 3** confère au robot une dimension proactive et sociale, lui permettant d'initier des interactions personnalisées dès la détection d'un utilisateur.

Ensemble, ces trois mécanismes — réactif, mémoriel et proactif — forment la fondation de l'expérience utilisateur unique que KODA ambitionne d'offrir : celle d'un compagnon intelligent, attentionné et toujours présent.